

Prompting Guide

Good Prompts vs. Bad Prompts · Claude Code + Bedrock AgentCore Hackathon · Energy Symposium

The biggest time sink in dry runs wasn't Claude Code — it was scope and ambiguity. This applies to any AI coding assistant, not just this hackathon: a vague prompt gets a vague result, an over-ambitious prompt gets a half-finished one, and an unverified assumption gets a silent bug. These seven pairs are about calibration, not syntax.

For what to install and which MCP servers to connect, see the separate **Technical Guide**. For terminology, see the **Glossary**. Both are on the hackathon portal.

1. Too vague → Specific and scoped

✗ *“Build me an agent for pipeline leaks.”*

✓ *“Read use-case-3’s one-pager and every file under data/. Propose one narrow question this agent could answer well using scada_timeseries.csv and the leak/false-positive labels. Then build a script — not yet an agent — that answers it and prints its reasoning. Don’t touch AgentCore yet.”*

A vague prompt forces the model to guess at scope. Naming the exact files and asking for a script before an agent keeps the first result small enough to check.

2. No structure (Track 2) → Names the pattern and the agents

✗ *“Build me a multi-agent system for predictive maintenance.”*

✓ *“Using the Strands SDK Agents as Tools pattern, build two specialized agents: a sensor-anomaly agent that flags deviations from vibration_baselines.csv, and a history agent that checks pump_maintenance_history.json for prior occurrences on that pump. Wire both into an orchestrator that combines them into one diagnostic. Check which Bedrock model ID is enabled in my lab account before hardcoding one.”*

Naming the orchestration pattern and each agent’s one job produces a design you can review before any code gets written.

3. Everything at once → Demo first, platform second

✗ *“Deploy this to production with full auth, a gateway, and a custom UI.”*

✓ *“Before adding Identity, Gateway, or a UI: get the agent answering one real question end-to-end, locally. Once that works, deploy it to AgentCore Runtime — that’s the only required module. Add Memory, Gateway, or Identity only if there’s time left after the demo works.”*

This is the single biggest risk we saw in dry runs — teams that started with the full platform build ran out of time before they had a working demo. Runtime is the only required AgentCore module; everything else is a stretch goal.

4. Blind join → Verify the join key first

✗ *“Join well_metadata to daily_drilling_reports.csv on well_id and summarize NPT by well.”*

✓ *“Before joining, print the first 5 values of well_id from both files and confirm they’re the same format. This dataset has two DDR corpora that don’t share a key — check which one you’re in before writing the join.”*

Several datasets in this hackathon have a join trap on purpose (mismatched ID formats, two corpora that don’t cross-join). A blind join returns zero rows silently — it doesn’t error, it just quietly gives you nothing.

5. Paste it all in → Aggregate first

✗ *“Here’s the sensor CSV, tell me which pump is about to fail.”* (pasting a 625,536-row file directly into the conversation)

✓ *“Write a script that computes rolling anomaly scores per pump from sensor_timeseries.csv and prints only the top 5 deviations. Don’t load the whole file into the conversation — it’s 625K rows / ~60MB.”*

A large file doesn’t fit in the model’s context window, and even if it technically did, it would crowd out everything else the model needs to reason about. Compute the aggregate in code; bring only the aggregate into the conversation. If you’re building a multi-agent pipeline, this problem compounds across agents — see Multi-Agent Context Management below.

6. Guess the API → Look it up first

✗ *“Call the Bedrock AgentCore API to create a memory resource.”* (letting the model guess parameter names)

✓ *“Use the AWS Documentation MCP server to confirm the exact AgentCore Memory API parameters before writing the call.”*

Model training data has a cutoff; AgentCore’s API shapes and current model IDs don’t. An MCP server that can look up current docs turns a guess into a verified answer — install instructions are in the Technical Guide.

7. Swallow the error → Fail loud

✗ *“If the join returns no rows, just skip it and move to the next file.”*

✓ *“If a join returns 0 rows, stop and print the first few values from each join column — a silent empty join usually means a format mismatch, not ‘no matching data.’”*

An agent that quietly tolerates a zero-row join or a caught-and-ignored exception will confidently report a wrong answer instead of a debuggable failure. Ask Claude Code to fail loud on anything unexpected, not swallow it.

Multi-Agent Context Management

If you’re on Track 2, this is the failure mode most likely to hit you and it won’t look like an error message — it’ll look like the orchestrator running out of context, giving a truncated or generic answer partway through a pipeline.

The datasets in this hackathon are large (UC2’s sensor file is 625K rows). Prompt pair 5 above covers not pasting a whole file into one turn — but that’s not the only place this bites. In a multi-agent pipeline (Agents as Tools, Graph, or Workflow), every tool result and every sub-agent’s output can get folded into the orchestrator’s context. A specialized agent that returns raw rows instead of a summary, called once per pump or once per station across a 12-pump or 8-station loop, will blow through the context window before the orchestrator ever produces an answer — even though no single call looked large on its own.

Four things that keep this from happening:

1. **Cap what a tool returns, not just what you ask for in a prompt.** A tool function that queries `sensor_timeseries.csv` should return a fixed-size result by construction — the top 5–10

anomalies as a short table, or a numeric summary — never every matching row. Enforce the cap in the tool's code, not by hoping the model self-regulates.

2. **Give each specialized agent a hard output budget in its system prompt.** Something concrete and checkable, e.g. *“Respond in under 150 words. Return structured findings only — no raw data, no restating the input.”* A vague “be concise” doesn’t reliably hold; a number does.
3. **Have the orchestrator work from summaries, not intermediate raw data.** If a sub-agent needs row-level detail to reach its conclusion, that detail should stay local to that agent’s own reasoning — only the conclusion travels upstream.
4. **Watch the growth as you add pipeline steps, not just at the start.** Context accumulates hop over hop. A pipeline that’s fine at 3 agents can overflow at 6 if each one appends its own uncapped output. Re-check total context size every time you add an agent, not just once at the beginning.

Connected MCP servers add to this too — every tool definition on every connected server sits in context for the whole session. See the note on the AgentCore MCP server in the Technical Guide.

Logging in, environment setup, and how to submit: see the hackathon portal. **What to install and which MCP servers to connect:** see the Technical Guide.